

DAT410 - Design of AI Systems

Group 69, Module 7		
Teammates	Jana Leong	Chung Fung Oi
Course	Bachelor of Computer Science	
Email	janale@student.chalmers.se	funngo@student.chalmers.se
No. of hours spent	18	

We hereby declare that we have both actively participated in solving every exercise. All solutions are entirely our own work, without having taken part in other solutions.

1. Reading and summary [Group]

Read and briefly summarise the following paper:

- [GUS - a frame-driven dialog system Links to an external site.](#)

This paper is an early work exhibiting several main ideas on which today's common digital assistants are based.

GUS (Genial Understander System), developed at Xerox PARC in the 1970s, represents a seminal advancement in conversational artificial intelligence. It moved the field beyond rudimentary keyword matching towards a more sophisticated frame-driven architecture, establishing a blueprint for modern digital assistants.

Architecture and Core Methodology

GUS is structured as a modular system wherein discrete processes—including morphological analysis, syntactic parsing, reasoning, and language generation—interact through an asynchronous control mechanism. Central to its operation is a dynamic agenda; the system evaluates the current conversational state to prioritize and execute the next logical task, ensuring a coherent flow of information.

Knowledge Frames

The primary cognitive component of the GUS is the Frame Reasoner, who is responsible for the system's domain knowledge in order to provide answers that fit into the context of the user. In this context, frames are complex data structures used to represent stereotypical knowledge of objects or events, such as "Trip", a "Date" or a "Person". Each frame consists of multiple slots which are placeholders for specific information that the system needs to "fill" during the chat interaction. These "fill-in-the-blanks" enable the system to set up small goals in the form of a template to reply to the user. For instance, a "Date" frame includes slots for Month, Day, and Year.

Syntactic Analyser

If the Knowledge Frame is the cognitive component, then the Syntactic Analyser (SA) is the first stage of the Language Analysis Pipeline. Its role, to resolve the linguistic "noise" of a sentence into a clean grammatical structure. While the Knowledge Frame "knows" what a trip is, it has no idea how to read a sentence like "I'd like to leave for San Diego **at around ten in the morning.**"

Its functional component takes raw, unorganized data and transforms it into a structured format such that the Frame Reasoner can actually use.

The Syntactic Analyzer breaks this down in to:

- The verb: “to leave for”
- The destination: “San Diego” (recognizing it as a prepositional phrase of location, an Object)
- The time: “around ten in the morning” (recognizing it as a temporal modifier of time)

In the GUS architecture, the SA works in tandem with a Case-Frame analyser. This crucial link bridges the gap between grammar and cognition.

- The SA says: “This noun is the object of the preposition ‘to leave for’ is to go somewhere.”
- The Case-Frame Analyser says: “Because the verb is ‘to go’ then San Diego would be the TO-PLACE, in specific the destination.”
- The Knowledge Frame then takes this 2 information that TO-PLACE and stores it in the destination slot.

It performs a similar mapping for the time fragment. By using this logic the system doesn’t need to ask “Where are you going?” since the user has already voluntarily provided the information.

Furthermore, unlike simpler systems where the analyser finishes its job and quits, the GUS SA is interactive. If a parse violates selectional restrictions (identified by the Case-Frame Analyser) or creates a contextual inconsistency (identified by the Frame Reasoner), a feedback signal is sent to the SA to re-examine the sentence for a different interpretation. The SA then utilizes its Transition-Network Grammar to explore alternative paths within the Chart, effectively “re-parsing” the input to find a semantically and contextually appropriate interpretation.

Ultimately, the Syntactic Analyser provides the **structure**, the Case-Frame Analyser provides the **meaning**, and the Knowledge Frame provides the **context**.

The Lexical Analyser and the “Chart”

Before the Syntactic Analyser can apply its grammar rules, the Lexical Analyser performs the initial “groundwork” on the user’s input, translating raw data into canonical data, using:

- Dictionary & Morphological Lookup: using a dictionary of over 3,000 stems and a set of morphological rules to identify word roots, suffixes, and prefixes. It identifies that words like “leaving” come from the stem word “leave”, or that “San Diego” should be treated as a single multi-word idiom rather than two separate words.
- The “Chart”: This output of this process is a specialised data structure that puts every word or phrase into a “well-informed table” of the sentence:
 - Syntactic categories: Is the word a noun, verb or preposition?
 - Semantic features: Is “San Diego” a city? Is “Monday” a day?
 - Potential ambiguities: Does this word have multi-meanings?

Asynchronous Control and Semantic Feedback

As previously mentioned the GUS analysis is non-static and does not operate in a vacuum. It is integrated into the system's Asynchronous Control Mechanism where:

- **State Preservation:** exists because the system is implemented using INTERLISP's coroutine facility.
- **Feedback Loop:** If the Case-Frame analysis finds that a parse violates selection restrictions or semantic roles, it can feed the information back to the parse. To allow for the syntactic analyser to "resume where it is suspended" to find a more semantically appropriate interpretation.

Servants and Demons

In order to fulfill the slot-filling through procedural attachment, GUS employs the help of servants and daemons.

- **Servants:** These are procedures activated on demand upon the need to retrieve missing information, either by querying an internal database or by prompting the user.
- **Demons:** These are autonomous procedures that trigger automatically once a specific slot is filled. They enable the system to perform secondary computations, such as resolving "next friday" into a specific calendar date or providing logical default values (e.g. assuming the current year if none is specified).

Managing Natural Dialogue

Conversations and natural language is inherently complex, given so, GUS was engineered to navigate the "vagaries" of human communication through several key capabilities:

- **Mixed Initiative:** The architecture supports a flexible dialogue where the user can volunteer information out of sequence, shift the topic, or ask unsolicited questions, rather than merely responding to the system prompts.
- **Indirect Answers:** The system maintains some level of inference in order to interpret indirect answers for simpler manifestations via the very narrow expectations about the subject matter and client's goals.
- **Anaphora Resolution:** The system maintains a discourse history to resolve linguistic references; for example, identifying that "the first one" refers to an earlier mentioned topic/ idea.
- **Resolution of Sentence Fragments:** By utilising "skeletons", GUS can interpret isolated fragments—such as the response "San Diego" and map them into a complete semantic thought based on the preceding context.

Domain Restriction and Realism

To ensure functional reliability and structural depth, the developers restricted GUS to the specific domain of a travel agent. By narrowing the discourse to a single objective, arranging a

return flight, the system could leverage a highly specialized “Dialog Frame” to effectively guide the user towards a completed reservation.

2. Implement a simple dialogue system [Group]

The purpose of this task is to make you aware of the basic steps and challenges involved in designing a digital assistant.

a) Implement a simple text-based digital assistant that can help with at least three things, for example:

- provide a weather forecast
- find a restaurant
- find the next tram/bus

However, implement in such a way that the assistant is extensible to a much larger set of tasks.

Note that it is possible to implement this in a trivial way by having the system fully control the flow like in "Enter 1 for account, 2 for insurance, 3 for loans", and continue in the same way with very specific questions. However, a dialogue system aims for a more flexible style, allowing for a more natural conversation. This is a non-trivial task, and while it is clearly not possible to fully solve in the scope of a weekly project, it is quite possible to get an understanding of many of the issues involved.

We suggest that you implement your assistant in the simplest possible way based on keyword matching (at least at first). Also think of the assistant as having access to back-end systems with the appropriate databases and functionality - for the purpose of this task you can simply use fictitious or random data.

As the output of this task, briefly describe how you have implemented the system, and especially any non-trivial functionality. Show some sample outputs, where the dialogue is as advanced as your system allows. Also show or discuss the limitations of your system. Submit your code in the report.

The assistant is a text-based dialogue system implemented in Python. It supports four primary capabilities: live weather forecasts, restaurant recommendations, public transport departure times, and directions between locations. The system is designed to be extensible, adding any new features requires only registering a new intent object without modifying any existing components of the system.

The assistant accepts free-text input and attempts to infer the user's intention through pattern recognition. Once an intent is identified, the system engages in conversational interaction, asking follow-up questions when required information is missing. This design allows the assistant to approximate a more natural conversation flow rather than forcing the user into pre-defined interaction paths.

Architecture

The dialogue system follows a modular architecture consisting of four layers, each responsible for a specific aspect of the interaction process.

Layer	Class	Responsibility
Data	fetch_weather(), RESTAURANT_DB, TRANSPORT_DB, DIRECTIONS_DB	Back-end data sources. Weather is retrieved from the Visual Crossing API, while restaurant, transport, and direction data are stored locally as simplified datasets.
Memory	ConversationContext	Maintains conversation state by storing extracted entities (such as city, cuisine, line number, origin, and destination) across dialogue turns. It also tracks the previously matched intent.
Matching	Intent, IntentRegistry	Each intent contains its own trigger patterns, extraction function, and response function. The registry stores all intents and returns the first matching one.
Orchestration	DialogueManager	Controls the overall dialogue flow by processing user input, selecting the appropriate intent, extracting relevant information, and generating responses.

This layered design separates responsibilities between components, making the system easier to maintain and extend.

How a Turn is Processed

Each user input is processed by the `DialogueManager.process()` method, which follows a consistent sequence of steps.

First, the system attempts to match the input against all registered intents using the `IntentRegistry.match()` function. This function scans each intent's patterns and returns the first match detected.

If a matching intent is found, the intent's `extract_fn` function is executed. This function uses regular expressions to extract relevant entities from the user input, such as city names or transport line numbers. Extracted values are stored in the `ConversationContext` as slots.

Next, the `respond_fn` function generates the system's response. If all required information is present in the context, a full answer is produced. If essential information is missing, the system instead generates a clarification question requesting the missing slot.

If no intent matches the new input, the system checks whether it is currently expecting a follow-up. This is done through `_expects_follow_up()`, which only returns true when the assistant is waiting for missing slot information. In that case, the assistant reruns the last matched intent using the new text. This allows short replies such as "London", "16", or "Central Station" to be interpreted correctly in context.

If neither direct matching nor follow-up resolution succeeds, the assistant returns a generic fallback message.

Intent Class

Each capability of the assistant is implemented as an `Intent` object containing four key components:

Field	Purpose
<code>name</code>	Unique identifier of the intent (e.g., "weather"). Used to recall the intent during follow-up interactions.
<code>patterns</code>	A list of keywords or regular expressions that activate the intent
<code>extract_fn</code>	Function that extracts slot values from the input and stores them in the context
<code>respond_fn</code>	Function that generates the system response based on the stored slots.

This design ensures that new capabilities can be added independently without modifying the core dialogue manager.

Implemented Capabilities

Weather forecasts

The weather skill supports live weather lookup for any city using the Visual Crossing Weather API. It accepts both explicit dates in the format DD-MM-YYYY and relative expressions such as “today”, “tomorrow”, “yesterday”, “day after tomorrow”, and even phrases such as “in 3 days”. If no date is provided, the system defaults to today.

The assistant first parses the date using `parse_weather_date()`, then attempts to extract the city name from patterns such as “weather in London” or “forecast for Berlin”. If the city is missing, it asks a follow-up question. The weather response includes condition, temperature, feels-like temperature, wind speed, and humidity.

The implementation also includes error handling for invalid API keys, invalid city names, bad requests, rate limits, server failures, connection errors, and timeouts. If a weather query fails, the city slot is cleared so the user can retry immediately with a corrected city name.

Restaurant recommendations

The restaurant skill searches a local hardcoded restaurant database. Each entry contains a restaurant name, cuisine, rating, area, price level, and opening hours. The assistant can filter results by cuisine and area, then rank them by rating.

To make interaction more natural, the system includes a cuisine synonym dictionary. For example, “sushi” is mapped to “japanese” and “pizza” is mapped to “italian”. This means the user does not need to use exactly the same labels as the internal database.

The restaurant extractor also detects unsupported cuisines and unsupported areas. Instead of failing silently, it informs the user which cuisines and areas are available in the demo. In addition, if the user makes a generic restaurant request without specifying cuisine or area, old slot values are cleared to prevent stale preferences from affecting the new search.

Public transport departures

The transport skill retrieves departure times from a local hardcoded tram and bus schedule. The assistant can answer questions such as “When is the next tram on line 16?” and returns upcoming departures, destinations, and platforms.

If the user asks about transport without specifying a line number, the assistant asks a follow-up question. If the user enters a line that is not present in the demo database, the assistant responds with a helpful message listing the supported lines.

As with the restaurant skill, stale line numbers are cleared for generic transport questions so that old context is not accidentally reused.

Directions between places

The directions skill uses a hardcoded route database for a small set of known locations. It supports queries such as “Directions from Central Station to Liseberg”, “Go to Lindholmen”, or even compact forms such as “Central Station to Hisingen”.

This skill is slightly more advanced because it tracks which specific slot is missing using `awaiting_direction_slot`. If the destination is missing, the assistant asks where the user wants to go. If the origin is missing, it asks where the user is starting from. This allows short follow-up answers to be assigned correctly.

The code also supports stale-slot invalidation for generic directions requests, ensuring that new route queries begin with a fresh state rather than reusing previous origin or destination values.

Sample Dialogues

Dialogue 1 — Two-turn weather query (slot filling)

You: What is the weather like?
Assistant: Which city would you like the weather for?
You: Gothenburg
Assistant: Weather in Gothenburg on 10-03-2026: Condition : Partially cloudy Temp : 2C (feels like 0C) Wind : 17 km/h Humidity : 88%

Dialogue 2 — One-shot weather with date

You: What is the weather like in London tomorrow?
Assistant: Weather in London on 11-03-2026: Condition : Rain, partially cloudy Temp : 10C (feels like 8C) Wind : 22 km/h Humidity : 71%

Dialogue 3 — Restaurant with follow-up cuisine filter

You: I am hungry, any restaurant suggestions?
Assistant: Top restaurants: Rakultur Japanese Rating: 4.8 \$\$\$ Open: 12:00-22:00 Smaka Swedish Rating: 4.6 \$\$ Open: 11:00-22:00 Dhaba Indian Rating: 4.5 \$\$ Open: 11:30-21:30
You: Japanese

Assistant: Top restaurants (japanese):	
Rakultur	Japanese Rating: 4.8 \$\$\$ Open: 12:00-22:00
Nori	Japanese Rating: 4.4 \$\$ Open: 11:00-21:00

Dialogue 4 — Transport two-turn

You: When is the next tram

Assistant: Which line? I have schedules for: 1, 2, 6, 11, 16, 55

You: 1

Assistant: Line 1 upcoming departures:
Towards Angered +4 min Platform A
Towards Angered +19 min Platform A

Dialogue 5 — Directions

You: Directions

Assistant: Where would you like directions to?

You: Liseberg

Assistant: Where are you starting from?
--

You: Central Station

Assistant: Directions from Central Station to Liseberg:
Duration : 12 min
Distance : 3.2 km
Take : tram 5 or 6

Non-Trivial Functionality

1. Slot Filling Across Turns

A central feature of the system is slot filling. The ConversationContext stores values extracted from previous turns, allowing the user to provide information step by step rather than in one complete sentence. For example, the assistant can first ask for a city, then continue the weather query once the user replies with only “London”.

2. Explicit follow-up gating

The updated system does not always reuse the last intent blindly. Instead, the `_expects_follow_up()` method checks whether the system is actually waiting for missing information. This makes the dialogue flow more robust, because the assistant only treats a short reply as a continuation when there is an unfinished request.

3. Cuisine Synonyms

The restaurant functionality includes a small synonym layer so that users can say “sushi” instead of “japanese” or “pizza” instead of “italian”. This improves usability without requiring more advanced NLP models.

4. Stale Slot Invalidation

A subtle issue in slot-filling systems is that old slot values from a previous query can bleed into a new, unrelated query. For example, if the user first asks about sushi restaurants and then asks “find me a restaurant” with no cuisine, a naive system would re-use the old sushi slot. The restaurant and transport intents explicitly clear stale slots when a generic (non-specific) query is detected, ensuring the new query is answered cleanly.

5. Directions Slot Tracking

The directions skill uses a dedicated state variable, `awaiting_direction_slot`, to keep track of whether the system is waiting for an origin or a destination. This avoids ambiguity in short follow-up replies and makes the route dialogue more accurate.

6. Live Weather API with Full Error Handling

The weather intent makes a real HTTP request to the Visual Crossing Weather API. The `fetch_weather()` function handles six distinct HTTP error codes (401, 404, 400, 429, 5xx) and also catches network-level exceptions (connection errors, timeouts), returning a structured error message in each case. When an error occurs, the city slot is cleared so the user can immediately retry with a different city name.

Limitations

Although the system demonstrates several useful dialogue system concepts, it also has limitations.

First, the intent recognition relies on keyword matching and predefined patterns. As a result, unexpected synonyms or unusual phrasing may not be recognized. For example, words like “eatery” or “grub” may fail to trigger the restaurant intent.

Second, the system has no real semantic understanding. It cannot handle negation properly, so “not sushi” may still be interpreted as a sushi-related request. It also cannot reason about more complex user goals or combine several tasks in one utterance.

Third, intent selection is based on first-match priority. This means that if more than one intent could apply, the assistant simply chooses whichever intent appears first in the registry rather than comparing confidence scores.

Fourth, the route, restaurant, and transport data are local demo datasets. They are sufficient for illustrating dialogue management, but they are not realistic enough for real-world deployment. Only the weather skill uses a live external API.

Finally, the assistant does not resolve pronouns or references such as “there”, “that one”, or “the second option”. It also does not personalise results, maintain long-term preferences, or perform advanced dialogue repair.

b) Suggest how - if more time were available - you could make your dialogue system more advanced.

Named entity recognition for slot extraction

Instead of using hand-written regular expressions, the system could use Named Entity Recognition to identify cities, places, dates, and route endpoints more robustly. This would reduce manual engineering and improve performance on unseen inputs.

Probabilistic dialogue state tracking

Currently, each slot stores a single fixed value. A more advanced system could maintain a belief state with multiple possible slot values and their probabilities. This would make it easier to handle ambiguity and uncertain input.

Real API integrations

The restaurant, transport, and directions functionalities could be replaced with real APIs. For example, restaurant search could use Google Places, public transport could use a live local transit API, and route planning could use a mapping or navigation service. This would make the assistant much more realistic and practically useful.

Reference resolution and richer context tracking

The dialogue context could be expanded to support pronouns, references to previous results, and richer conversational memory. This would allow users to say things like “What about the second one?” or “How do I get there from here?”.

module7group69

March 10, 2026

```
[ ]: import re
import requests
from datetime import datetime, timedelta

# =====
# CONFIGURATION
# =====
VISUAL_CROSSING_API_KEY = "V6LAWM6D8QKAEKTKPW2MN4DCV"

# =====
# BACK-END DATA
# =====

def parse_weather_date(text):
    """
    Parse weather date from free text.
    Supports DD-MM-YYYY plus common relative words.
    Returns (date_obj_or_none, error_or_none).
    """
    t = text.lower()

    # Check explicit date first.
    m = re.search(r"\b(\d{2}-\d{2}-\d{4})\b", text)
    if m:
        try:
            return datetime.strptime(m.group(1), "%d-%m-%Y").date(), None
        except ValueError:
            return None, "Invalid date format. Please use DD-MM-YYYY."

    today = datetime.today().date()
    m_rel = re.search(r"\bin\s+(\d{1,2})\s+days?\b", t)
    if m_rel:
        days = int(m_rel.group(1))
        return today + timedelta(days=days), None
    if "day after tomorrow" in t:
        return today + timedelta(days=2), None
    if "tomorrow" in t:
```

```

        return today + timedelta(days=1), None
    if "yesterday" in t:
        return today - timedelta(days=1), None
    if "today" in t:
        return today, None

    # Default: today when no date is provided.
    return today, None

def fetch_weather(city, target_date):
    """
    Calls the Visual Crossing Weather API for a specific date.
    Returns a dict with weather info, or a dict with an 'error' key.
    Works for any city in the world.
    """
    date1 = target_date.strftime("%Y-%m-%d")
    date2 = date1
    url = (
        "https://weather.visualcrossing.com/VisualCrossingWebServices/rest/
↪services/timeline/"
        f"{city}/{date1}/{date2}?key={VISUAL_CROSSING_API_KEY}"
    )
    params = {
        "unitGroup": "metric",
        "include": "days",
        "contentType": "json",
    }
    try:
        r = requests.get(url, params=params, timeout=5)
        if r.status_code == 401:
            return {"error": "Invalid API key. Check VISUAL_CROSSING_API_KEY."}
        if r.status_code == 404:
            return {"error": f'City "{city}" not found.'}
        if r.status_code == 400:
            return {
                "error": (
                    f'I could not find a valid city for "{city}". '
                    "Please enter a specific city, for example: 'Weather in
↪Gothenburg'."
                )
            }
        if r.status_code == 429:
            return {"error": "Weather service rate limit reached. Please try
↪again in a moment."}
        if r.status_code >= 500:

```

```

        return {"error": "Weather service is temporarily unavailable.␣
↪Please try again later."}
        r.raise_for_status()
        d = r.json()
        if "days" not in d or not d["days"]:
            return {"error": "No weather data available for the requested date.
↪"}

        day = d["days"][0]
        return {
            "temp": round(float(day.get("temp", 0))),
            "feels": round(float(day.get("feelslike", day.get("temp", 0)))),
            "condition": str(day.get("conditions", "Unknown")).capitalize(),
            "wind": f"{round(float(day.get('windspeed', 0)))} km/h",
            "humidity": f"{round(float(day.get('humidity', 0)))}%",
            "date": target_date.strftime("%d-%m-%Y"),
        }
    except requests.exceptions.ConnectionError:
        return {"error": "No internet connection."}
    except requests.exceptions.Timeout:
        return {"error": "Weather request timed out. Please try again."}
    except Exception as e:
        return {"error": "Weather lookup failed due to an unexpected error.␣
↪Please try a different city."}

# Hardcoded restaurant database
RESTAURANT_DB = [
    {"name": "Smaka",          "cuisine": "swedish", "rating": 4.6, "area":␣
↪"central",    "price": "$$", "hours": "11:00-22:00"},
    {"name": "Rakultur",      "cuisine": "japanese", "rating": 4.8, "area":␣
↪"central",    "price": "$$$", "hours": "12:00-22:00"},
    {"name": "Mr Pizza",      "cuisine": "italian",   "rating": 4.2, "area":␣
↪"hisingen",   "price": "$",   "hours": "10:00-23:00"},
    {"name": "Dhaba",         "cuisine": "indian",    "rating": 4.5, "area":␣
↪"lindholmen", "price": "$$",  "hours": "11:30-21:30"},
    {"name": "Tacos & Tequila", "cuisine": "mexican", "rating": 4.3, "area":␣
↪"central",    "price": "$$",  "hours": "12:00-00:00"},
    {"name": "Nori",          "cuisine": "japanese", "rating": 4.4, "area":␣
↪"hisingen",   "price": "$$",  "hours": "11:00-21:00"},
    {"name": "Punjab Palace", "cuisine": "indian",   "rating": 4.3, "area":␣
↪"central",    "price": "$$",  "hours": "12:00-22:00"},
]

# Hardcoded tram/bus schedule
TRANSPORT_DB = {

```



```

    "1": [{"dest": "Angered", "dep": "+4 min", "platform": "A"},
          {"dest": "Angered", "dep": "+19 min", "platform": "A"}],
    "2": [{"dest": "Molndal", "dep": "+2 min", "platform": "B"},
          {"dest": "Molndal", "dep": "+17 min", "platform": "B"}],
    "6": [{"dest": "Lansmangarden", "dep": "+8 min", "platform": "C"},
          {"dest": "Lansmangarden", "dep": "+23 min", "platform": "C"}],
    "11": [{"dest": "Saltholmen", "dep": "+6 min", "platform": "D"},
           {"dest": "Saltholmen", "dep": "+21 min", "platform": "D"}],
    "16": [{"dest": "Hoegsbo", "dep": "+3 min", "platform": "A"},
           {"dest": "Hoegsbo", "dep": "+18 min", "platform": "A"}],
    "55": [{"dest": "Angered C.", "dep": "+1 min", "platform": "E"},
           {"dest": "Angered C.", "dep": "+16 min", "platform": "E"}],
}

# Hardcoded directions between known locations
DIRECTIONS_DB = {
    ("central station", "liseberg"): {"duration": "12 min",
    ↪ "distance": "3.2 km", "mode": "tram 5 or 6"},
    ("central station", "lindholmen"): {"duration": "8 min",
    ↪ "distance": "2.1 km", "mode": "ferry or bus 16"},
    ("central station", "hisingen"): {"duration": "15 min",
    ↪ "distance": "4.5 km", "mode": "bus 16 or 32"},
    ("central station", "gothenburg university"): {"duration": "20 min",
    ↪ "distance": "5.1 km", "mode": "tram 13"},
    ("liseberg", "central station"): {"duration": "12 min",
    ↪ "distance": "3.2 km", "mode": "tram 5 or 6"},
    ("lindholmen", "central station"): {"duration": "8 min",
    ↪ "distance": "2.1 km", "mode": "ferry or bus 16"},
}

def lookup_directions(origin, destination):
    key = (origin.lower().strip(), destination.lower().strip())
    return DIRECTIONS_DB.get(key) or DIRECTIONS_DB.get((key[1], key[0]))

# =====
# CONVERSATION CONTEXT (slot store)
# =====
class ConversationContext:
    """
    Persists named slots across turns, e.g. city='london', cuisine='sushi'.
    Also tracks the last matched intent for follow-up resolution.

    Example
    -----
    Bot: "Which city?"
    User: "London"          <- no intent match, but last_intent='weather'

```

```

so DialogueManager re-runs weather with 'London'

"""
def __init__(self):
    self.slots = {}
    self.last_intent = None
    self.turn = 0

def set(self, key, value): self.slots[key] = value
def get(self, key): return self.slots.get(key)
def clear(self, key): self.slots.pop(key, None)
def clear_all(self): self.slots.clear()

def remember(self, intent_name):
    self.last_intent = intent_name
    self.turn += 1

def __repr__(self):
    return f"Context(turn={self.turn}, last={self.last_intent}, slots={self.slots})"

# =====
# INTENT & INTENT REGISTRY
# =====
class Intent:
    """
    One capability of the assistant.

    Parameters
    -----
    name : str
    patterns : list of str (substring) or compiled regex objects
    extract_fn : (text, ctx) -> None pulls slot values from user input
    respond_fn : (ctx) -> str builds reply from current slots
    """
    def __init__(self, name, patterns, extract_fn, respond_fn):
        self.name = name
        self.patterns = patterns
        self.extract = extract_fn
        self.respond = respond_fn

    def matches(self, text):
        t = text.lower()
        for p in self.patterns:
            if isinstance(p, str):
                # Use token-aware matching for literal phrases so
                # short strings like "hi" do not match inside words

```

```

        # such as "hisingen".
        if re.search(rf"\b{re.escape(p.lower())}\b", t):
            return True
    if hasattr(p, "search") and p.search(t):
        return True
    return False

class IntentRegistry:
    """
    Holds all intents. Add a new skill via register() - nothing else changes.
    Matching is a linear scan; first match wins.
    """
    def __init__(self):
        self._intents = []
    def register(self, intent):
        self._intents.append(intent)
    def names(self):
        return [i.name for i in self._intents]

    def match(self, text):
        for i in self._intents:
            if i.matches(text): return i
        return None

    def get_by_name(self, name):
        for i in self._intents:
            if i.name == name: return i
        return None

# =====
# INTENT DEFINITIONS
# =====
registry = IntentRegistry()

# GREETING
registry.register(Intent(
    name="greeting",
    patterns=["hello", "hi", "hey", "good morning", "good evening", "howdy"],
    extract_fn=lambda t, c: None,
    respond_fn=lambda c: (
        "Hello! I can help you with:\n"
        " - Weather forecasts (live data)\n"
        " - Restaurant suggestions\n"
        " - Tram/bus departure times\n"
        " - Directions between places\n"
        "Type 'help' for example questions."
    ),
))

```

```

# WEATHER
def weather_extract(text, ctx):
    parsed_date, date_error = parse_weather_date(text)
    if date_error:
        ctx.set("date_error", date_error)
    else:
        ctx.set("date", parsed_date)
        ctx.clear("date_error")

    # Remove date tokens/keywords before extracting city so
    # inputs like "weather in gothenburg 11-03-2026" are handled.
    city_source = re.sub(r"\b\d{2}-\d{2}-\d{4}\b", "", text, flags=re.
↪IGNORECASE)
    city_source = re.sub(r"\bin\s+\d{1,2}\s+days?\b", "", city_source, flags=re.
↪IGNORECASE)
    city_source = re.sub(
        r"\b(today|tomorrow|yesterday|day after tomorrow)\b",
        "",
        city_source,
        flags=re.IGNORECASE,
    )

    m = re.search(r"\b(?:in|for|at)\b\s+([a-zA-Z\s]+)(?:\?|\$|,)", city_source,
↪re.IGNORECASE)
    if m:
        candidate = m.group(1).strip()
        if candidate and not re.search(r"\b(weather|forecast|temperature)\b",
↪candidate, re.IGNORECASE):
            ctx.set("city", candidate)
            return

    # If this is a generic weather question with no location, force slot filling
    # instead of reusing a previously remembered city.
    if re.search(r"\b(weather|forecast|temperature)\b", text, re.IGNORECASE):
        ctx.clear("city")
        return

    # Follow-up support: accept plain city names like "London".
    if re.
↪search(r"\b(weather|forecast|temperature|raining|sunny|cold|hot|today|tomorrow|yesterday)\b",
↪text, re.IGNORECASE):
        return
    cleaned = text.strip().strip("?.,")
    if re.fullmatch(r"[A-Za-z][A-Za-z\s\-']{0,49}", cleaned):
        ctx.set("city", cleaned)

```

```

def weather_respond(ctx):
    date_error = ctx.get("date_error")
    if date_error:
        return date_error

    city = ctx.get("city")
    if not city:
        return "Which city would you like the weather for?"
    target_date = ctx.get("date") or datetime.today().date()
    result = fetch_weather(city, target_date)
    if "error" in result:
        ctx.clear("city")
        return f"Weather lookup failed: {result['error']}"
    return (
        f"Weather in {city.title()} on {result['date']}: \n"
        f" Condition : {result['condition']} \n"
        f" Temp      : {result['temp']}C (feels like {result['feels']}C) \n"
        f" Wind       : {result['wind']} \n"
        f" Humidity    : {result['humidity']} \n"
    )

registry.register(Intent(
    name="weather",
    patterns=[re.compile(r"weather|forecast|temperature|raining|sunny|cold|
↳outside|hot outside")],
    extract_fn=weather_extract,
    respond_fn=weather_respond,
))

# RESTAURANT
CUISINES = ["japanese", "italian", "indian", "swedish", "mexican"]
# Map common user terms to cuisines supported by the demo data.
CUISINE_SYNONYMS = {
    "sushi": "japanese",
    "pizza": "italian",
    "mexico": "mexican",
    "india": "indian",
    "sweden": "swedish",
}
AREAS = ["central", "hisingen", "lindholmen"]

def restaurant_extract(text, ctx):
    t = text.lower()
    found_cuisine = False
    found_area = False
    ctx.clear("invalid_cuisine")
    ctx.clear("invalid_area")

```

```

for alias, canonical in CUISINE_SYNONYMS.items():
    if re.search(rf"\b{re.escape(alias)}\b", t):
        ctx.set("cuisine", canonical)
        found_cuisine = True
for c in CUISINES:
    if re.search(rf"\b{re.escape(c)}\b", t):
        ctx.set("cuisine", c)
        found_cuisine = True
for a in AREAS:
    if re.search(rf"\b{re.escape(a)}\b", t):
        ctx.set("area", a)
        found_area = True

# If user specifies "<word> food" or "eat <word>" and it is unknown,
# return a helpful available-cuisines message instead of generic output.
cuisine_guess = None
m_food = re.search(r"\b([a-z]+\s+food\b", t)
if m_food:
    cuisine_guess = m_food.group(1)
else:
    m_eat = re.search(r"\beat\s+([a-z]+\b", t)
    if m_eat:
        cuisine_guess = m_eat.group(1)

known_cuisine_terms = set(CUISINES) | set(CUISINE_SYNONYMS.keys())
if cuisine_guess and cuisine_guess not in known_cuisine_terms:
    ctx.set("invalid_cuisine", cuisine_guess)

# If user says "in <area>" and area is unknown, provide valid options.
m_area = re.search(r"\bin\s+([a-z\s]+)?(?:\?|\$|,)", t)
if m_area and not found_area:
    area_guess = m_area.group(1).strip()
    if area_guess and area_guess not in AREAS:
        ctx.set("invalid_area", area_guess)

generic_query = bool(re.search(r"\b(top restaurants?|restaurants?
↪|food|eat|hungry|dinner|lunch)\b", t))
if generic_query and not found_cuisine:
    ctx.clear("cuisine")
if generic_query and not found_area:
    ctx.clear("area")

def restaurant_respond(ctx):
    invalid_cuisine = ctx.get("invalid_cuisine")
    if invalid_cuisine:
        ctx.clear("invalid_cuisine")

```

```

        available_cuisines = ", ".join(sorted(CUISINES))
        return (
            f'I do not have cuisine "{invalid_cuisine}" in this demo. '
            f"Available cuisines are: {available_cuisines}."
        )

    invalid_area = ctx.get("invalid_area")
    if invalid_area:
        ctx.clear("invalid_area")
        available_areas = ", ".join(AREAS)
        return (
            f'I do not have area "{invalid_area}" in this demo. '
            f"Available areas are: {available_areas}."
        )

    results = list(RESTAURANT_DB)
    cuisine = ctx.get("cuisine")
    area = ctx.get("area")
    if cuisine: results = [r for r in results if r["cuisine"] == cuisine]
    if area: results = [r for r in results if r["area"] == area]
    if not results:
        return "No restaurants found - try a different cuisine or area."
    top = sorted(results, key=lambda r: r["rating"], reverse=True)[:3]
    label = f" ({cuisine})" if cuisine else ""
    lines = [f"Top restaurants{label}:"]
    for r in top:
        lines.append(
            f" {r['name']:20s} {r['cuisine'].title():8s} "
            f"Rating: {r['rating']} {r['price']} Open: {r['hours']}"
        )
    return "\n".join(lines)

registry.register(Intent(
    name="restaurant",
    patterns=[re.
    ↪compile(r"restaurant|eat|food|hungry|dinner|lunch|italian|japanese|indian|swedish|mexican"),
    extract_fn=restaurant_extract,
    respond_fn=restaurant_respond,
))

# TRANSPORT
def transport_extract(text, ctx):
    t = text.lower()
    found_line = False
    ctx.clear("invalid_line")
    m = re.search(r"\b(\d{1,2})\b", text)
    if m and m.group(1) in TRANSPORT_DB:

```

```

        ctx.set("line", m.group(1))
        found_line = True
    elif m:
        ctx.set("invalid_line", m.group(1))

    # Generic transport requests should not reuse stale line values.
    generic_query = bool(re.
↳search(r"\b(tram|bus|line|next|departure|arrive|transit)\b", t))
    if generic_query and not found_line:
        ctx.clear("line")

def transport_respond(ctx):
    invalid_line = ctx.get("invalid_line")
    if invalid_line:
        ctx.clear("invalid_line")
        available = ", ".join(sorted(TRANSPORT_DB.keys(), key=int))
        return f'I do not have line {invalid_line}. Available lines are:␣
↳{available}'

    line = ctx.get("line")
    if not line:
        available = ", ".join(sorted(TRANSPORT_DB.keys(), key=int))
        return f"Which line? I have schedules for: {available}"
    deps = TRANSPORT_DB[line]
    lines = [f"Line {line} upcoming departures:"]
    for d in deps:
        lines.append(f"    Towards {d['dest']:20s} {d['dep']:8s} Platform␣
↳{d['platform']}")
    return "\n".join(lines)

registry.register(Intent(
    name="transport",
    patterns=[re.compile(r"tram|bus|line \d+|next.*\d+|departure|when.
↳*arrive|transit")],
    extract_fn=transport_extract,
    respond_fn=transport_respond,
))

# DIRECTIONS
def directions_extract(text, ctx):
    t = text.lower()
    generic_direction_request = bool(re.
↳search(r"\b(direction|directions|route|navigate|get to|go to|help)\b", t))

    # Explicit "directions from X" form: set origin and ask destination.
    m_from_only = re.search(r"^(?:directions?\s+)?from\s+(.+?)(?:\?|\$)", t)
    if m_from_only and " to " not in t:

```



```

    ctx.set("origin", m_from_only.group(1).strip())
    ctx.clear("destination")
    ctx.set("awaiting_direction_slot", "destination")
    return

# Compact pair form: "<origin> to <destination>".
# This supports replies like "central station to hisingen".
compact_pair = re.search(r"^s*(.+?)\s+to\s+(.+?)\s*(?:\?|\$)", t)
if (
    compact_pair
    and not t.startswith("to ")
    and not re.
    match(r"^s*(direction|directions|route|navigate|get|go|help)\b", t)
):
    ctx.set("origin", compact_pair.group(1).strip())
    ctx.set("destination", compact_pair.group(2).strip())
    ctx.clear("awaiting_direction_slot")
    return

# If we explicitly asked for a missing slot, treat plain input as that slot.
awaiting = ctx.get("awaiting_direction_slot")
if awaiting in ("origin", "destination"):
    cleaned = t.strip().strip("?!,")
    if cleaned and re.fullmatch(r"[a-z0-9][a-z0-9\s\-\.\']{0,79}", cleaned):
        ctx.set(awaiting, cleaned)
        ctx.clear("awaiting_direction_slot")
        return

m = re.search(r"from\s+(.+?)\s+to\s+(.+?)\s*(?:\?|\$)", t)
if m:
    ctx.set("origin", m.group(1).strip())
    ctx.set("destination", m.group(2).strip())
    ctx.clear("awaiting_direction_slot")
    return

m = re.search(r"to\s+(.+?)\s+from\s+(.+?)\s*(?:\?|\$)", t)
if m:
    ctx.set("destination", m.group(1).strip())
    ctx.set("origin", m.group(2).strip())
    ctx.clear("awaiting_direction_slot")
    return

m = re.search(r"(?:directions?|get|go|navigate)\s+to\s+(.+?)\s*(?:\?|\$)", t)
if m:
    # New destination-only request: clear stale origin from previous turns.
    ctx.set("destination", m.group(1).strip())
    ctx.clear("origin")
    ctx.clear("awaiting_direction_slot")
    return

```

```

# Follow-up shorthand after "Where would you like directions to?"
if (
    re.fullmatch(r"[a-z0-9][a-z0-9\s\-\.\']{0,79}", t.strip().strip("?!.,"))
    and not ctx.get("destination")
    and not generic_direction_request
):
    ctx.set("destination", t.strip().strip("?!.,"))
    ctx.clear("awaiting_direction_slot")
    return

# Generic directions request (e.g., "direction help") should start a fresh
↳ frame
# instead of reusing stale slots from previous route queries.
if generic_direction_request:
    ctx.clear("origin")
    ctx.clear("destination")
    ctx.set("awaiting_direction_slot", "destination")

def directions_respond(ctx):
    dest = ctx.get("destination")
    origin = ctx.get("origin")
    if not dest:
        ctx.set("awaiting_direction_slot", "destination")
        return "Where would you like directions to?"
    if not origin:
        ctx.set("awaiting_direction_slot", "origin")
        return "Where are you starting from?"
    ctx.clear("awaiting_direction_slot")
    d = lookup_directions(origin, dest)
    if not d:
        return (
            f'Sorry, I don\'t have directions from "{origin}" to "{dest}".\n'
            "Known places: Central Station, Liseberg, Lindholmen, Hisingen,
↳ Gothenburg University."
        )
    return (
        f'Directions from {origin.title()} to {dest.title()}: \n'
        f" Duration : {d['duration']} \n"
        f" Distance : {d['distance']} \n"
        f" Take      : {d['mode']} \n"
    )

registry.register(Intent(
    name="directions",
    patterns=[

```

```

        re.compile(r"direction|how (do i|to) get|route to|navigate|get to|go_
↳to"),
        re.compile(r"^s*[a-z0-9\s\-\.\']+\s+to\s+[a-z0-9\s\-\.\']+\s*$"),
        re.compile(r"^s*(?:directions?\s+)?from\s+.\s*$"),
    ],
    extract_fn=directions_extract,
    respond_fn=directions_respond,
))

# HELP
registry.register(Intent(
    name="help",
    patterns=["help", "what can you do", "capabilities", "options"],
    extract_fn=lambda t, c: None,
    respond_fn=lambda c: (
        "Here is what I can do:\n"
        "  Weather      : 'What is the weather in London?'\n"
        "  Restaurant   : 'Find me a sushi restaurant'\n"
        "               : 'Any indian food in Hisingen?'\n"
        "  Transport    : 'When is the next tram on line 16?'\n"
        "  Directions   : 'Directions from Central Station to Liseberg'\n"
        "You can ask in two steps - I remember what you said!"
    ),
))

# THANKS
registry.register(Intent(
    name="thanks",
    patterns=["thank", "thanks", "cheers", "great", "perfect", "awesome"],
    extract_fn=lambda t, c: None,
    respond_fn=lambda c: "You are welcome! Anything else?",
))

# =====
# DIALOGUE MANAGER
# =====
class DialogueManager:
    """
    Processes one user turn at a time.

    Flow
    ----
    1. Match an intent from the registry.
    2. If matched -> extract slots, generate response, save intent name.
    3. If no match -> follow-up resolution: re-run the last intent's
        extract + respond with the new input. Handles cases like:

```

```

        Bot: "Which city?" -> User: "London"
        Bot: "Which line?" -> User: "16"
4. If still nothing -> generic fallback.
"""
def __init__(self, registry):
    self.registry = registry
    self.ctx = ConversationContext()

def _expects_follow_up(self):
    """
    Returns True only when the assistant is explicitly waiting for
    missing slot information from the user.
    """
    if self.ctx.last_intent == "weather":
        return not self.ctx.get("city")
    if self.ctx.last_intent == "transport":
        return not self.ctx.get("line")
    if self.ctx.last_intent == "directions":
        return bool(self.ctx.get("awaiting_direction_slot")) or not self.
    ↪ ctx.get("destination") or not self.ctx.get("origin")
    return False

def process(self, text):
    text = text.strip()
    normalized_text = text.lower()
    intent = self.registry.match(normalized_text)

    if intent:
        intent.extract(normalized_text, self.ctx)
        reply = intent.respond(self.ctx)
        self.ctx.remember(intent.name)
        return reply

    # Follow-up: user is likely answering a clarification question
    if self.ctx.last_intent and self._expects_follow_up():
        last = self.registry.get_by_name(self.ctx.last_intent)
        if last:
            last.extract(normalized_text, self.ctx)
            reply = last.respond(self.ctx)
            self.ctx.remember(last.name)
            return reply

    return (
        "I didn't quite get that. "
        "Try asking about weather, restaurants, transport, or directions. "
        "Type 'help' for examples."
    )

```

```

def reset(self):
    self.ctx = ConversationContext()

# =====
# MAIN CHAT LOOP
# =====
def run():
    dm = DialogueManager(registry)
    print("=" * 60)
    print(" Digital Assistant (quit to exit | reset to clear memory)")
    print("=" * 60)
    print(f"Assistant: {dm.process('hello')}\n")

    while True:
        try:
            user = input("You: ").strip()
        except (EOFError, KeyboardInterrupt):
            print("\n[Session ended]")
            break

        if not user:
            continue
        if user.lower() in ("quit", "exit"):
            print("Assistant: Goodbye!")
            break
        if user.lower() == "reset":
            dm.reset()
            print("Assistant: Memory cleared. Starting fresh!\n")
            continue

        print(f"Assistant: {dm.process(user)}\n")

if __name__ == "__main__":
    run()

```